

Set

A Python set is an unordered collection of unique, immutable items.

- The **set itself is mutable** you are able to add and remove items from a set.
- **Elements must be hashable** (e.g., int, float, str or tuple with hashable items).
- **Non-hashable type** (list, dict or set) cannot be stored in a set.
- **No duplicates are allowed**: if you add the same item twice, it is stored only once.
- **Unordered**: items have no fixed position, so you cannot use indexing or slicing.
- An **empty set** is displayed as `set()`. (not `{}` - that refers to an empty dictionary!)

Contents:

1. [Creating a set](#)
 - Using the `set()` constructor
 - From other iterables
 - Simple assignment
 - Using set comprehension
2. [Common Operations](#)
 - `len()`
 - Adding and removing an item
 - printing all the values
3. [Set methods](#)
 - Mathematical operations: union, intersection, difference, symmetric_difference
 - Relational checks: `issubset`, `issuperset`, `isdisjoint`
 - Mutation methods: `add`, `clear`, `copy`, `discard`, `pop`, `remove`, `update`
4. [Summary](#)

Creating a set

Using the constructor

Calling `set()` with no arguments creates an empty set.

```
# set creation using constructor
a = set()
print(a)
print(type(a))
print(len(a))
```

From other iterables

Lecture 07

Passing an iterable to the set constructor will create a new set with all items of the iterable. Whenever you create or update a set, duplicates are automatically removed.

```
# set creation from an iterable
string_data = 'Hello World'
list_data = [x for x in range(10)]
tuple_data = (1, 2, 3, 2, 1, 3, 2)
range_data = range(10)
dict_data = {'zero': 0, 'one': 1, 'two': 2, 'three': 3}

print(set(string_data))
print(set(list_data))
print(set(tuple_data))
print(set(range_data))
print(set(dict_data))
# notice that the set does not contain duplicate characters
# and the order is not preserved
```

A set may not contain any non-hashable type

Non-hashable type like list, dict and set may not be in a set

```
# the following will fail because a list is not hashable
# a = {[3, 4]} # may not contain a list
# a = {1, 2, {3, 4}} # may not contain a set
# a = {1, 2, {'a':3, 'b':4}} # may not contain a dict

# if you uncomment the above lines, you will see a TypeError
```

From simple assignment

```
# simple assignment
a = {1, 2}
print(a)

a = {1, 2, (3, 1), 4, 'Hello'}
print(a)
a.clear()
print(a)
```

Set comprehension

This is a compact and efficient way of creating a set.

```
#set comprehension
#simple example
b = {x for x in range(10)}
print(b)

# set comprehension with condition
b = {x * x for x in range(10) if x % 2 == 0}
print(b)

# set comprehension with string
letters = {char.upper() for char in 'banana'}
```

Lecture 07

```
print(letters)  # {'B', 'A', 'N'}
```

#a more complex example

```
pairs = {(x, y) for x in range(3) for y in range(2)}  
print(pairs)
```

Common Operations

```
a.update({2, 3, 4})  
print(a)
```

Set specific methods

Returns a new set

- `difference()` - returns the item of first set AFTER the items of the second set are removed
- `intersection()` - returns the items that are common in both sets
- `union()` - return all the items in both sets

Returns a bool

- `isdisjoint()` -
- `issubset()` - returns True if all the items in the first set are contained in the second set
- `issuperset()` - returns True if all the items in the second set are contained in the first set

```
a = {2, 4, 6, 8}  
b = {1, 3, 5, 7, 9}  
c = {1, 2, 3, 4}  
d = {1, 2, 3, 4, 5, 6, 7, 8, 9}  
print(f'  a difference c: {a.difference(c)}')  
print(f'a intersection c: {a.intersection(c)}')  
print(f'      a union c: {a.union(c)}')  
print(f'      a union b: {a.union(b)}')  
print(f'  a isdisjoint b: {a.isdisjoint(b)}')  
print(f'  a isdisjoint c: {a.isdisjoint(c)}')  
print(f'  a issubset d: {a.issubset(d)}')  
print(f'  a issubset b: {a.issubset(b)}')  
print(f'  d issuperset c: {d.issuperset(c)}')  
print(f'  b issuperset c: {b.issuperset(c)}')
```

More set specific methods

- `add(item)` - adds a single item.
- `clear()` - removes all the items.
- `copy()` - returns a shallow copy.
- `discard(item)` - removes an element. No error even if it is not present.
- `pop()` - removes and return an arbitrary element.
- `remove(item)` - removes an element. If not present raises a `KeyError`.
- `update(iterable)` - add multiple elements from another iterable.

Lecture 07

```
#add(item) demo
tmp = {2, 4, 6, 8}
item = 5
print(f'Before: {tmp}', end=' ')
tmp.add(item)
print(f'-> after add: {item} -> {tmp}')

#clear() demo
tmp = {2, 4, 6, 8}
print(f'Before {tmp}', end=' ')
tmp.clear()
print(f'after clear -> {tmp}')

#copy() demo
tmp = {2, 4, 6, 8}
tmp_copy = tmp.copy()
print(f'Before {tmp}', end=' ')
tmp.clear()
print(f'after clear -> {tmp}')

#discard(item) demo
tmp = {2, 4, 6, 8}
item = 2
print(f'Before {tmp}', end=' ')
tmp.discard(item)
print(f'after discard {item} -> {tmp}')

# although the item is not in the set, there is no error
tmp = {2, 4, 6, 8}
item = 5
print(f'Before {tmp}', end=' ')
tmp.discard(item)
print(f'after discard {item} -> {tmp}')

#pop() demo
tmp = {2, 4, 6, 8}
item = 5
print(f'Before {tmp}', end=' ')
item = tmp.pop()
print(f'after pop {item} -> {tmp} <-> pop item: {item}')

#remove(item) demo
tmp = {2, 4, 6, 8}
item = 2
print(f'Before {tmp}', end=' ')
tmp.remove(item)
print(f'after remove {item} -> {tmp}')

# because the item is not in the set, there will be a KeyError
# tmp = {2, 4, 6, 8}
# item = 5
# print(f'Before {tmp}', end=' ')
# tmp.remove(item)
# print(f'after remove {item} -> {tmp}')

#update(item) demo
tmp = {2, 4, 6, 8}
iter = [1, 2, 5]
print(f'Before {tmp}', end=' ')
```

Lecture 07

```
tmp.update(iter)
print(f'after update {iter} -> {tmp}')
```

Summary

- A set is an unordered, mutable collection of unique hashable items
 - Allowed element types: (int, float, str, tuples (only if all their elements are hashable), bool, frozenset or bytes)
 - Not allowed: list, set, dict or bytearray
 - Operations: union, intersection, difference, symmetric difference.
 - Fast membership testing with `in`.
 - Useful for
 - Removing duplicates
 - Mathematical set operations
 - Efficient membership checks
-

Dictionary

A dictionary (`dict`) is an unordered, mutable collection of key-value pairs.

- Key must be unique and hashable object (e.g., tuples, strings and numbers).
- Value can be of any type
- List, set or other dictionaries cannot be used as keys because they are mutable

Contents:

4. [Creating a dictionary](#)
 - Literal assignment
 - `dict()` constructor
 - List of Tuples
 - `zip()` function
 - Dictionary comprehension
5. [Common Operations](#)
 - using `len()` to obtain the number of pairs
 - Accessing, updating, and deleting pairs
 - Iterating of keys and values
6. [More dict methods](#)
7. [Summary](#)

1. Creating a dict

Lecture 07

8. Via a literal assignment
9. Using the dict constructor
10. Using a list of tuples
11. Using the zip() function
12. Using dictionary comprehension (advanced)

```
# Literal assignment is probably the easiest way to get a dict
# You can use curly braces `{}` and separate key-value pairs with a
colon `:`.
a = {'ilia': 54, 'hao': 64, 'arben' : 75, 'mayy' : 70, 'yin' : 80,
'vinay' : 69}

# you can display the dictionary, via the print function
print(a)
```

Using the dict() constructor, this creates an empty dictionary

```
# creating an empty dictionary
a = dict()
print(a)

# from keyword arguments
a = dict(ilia=54, hao=64, arben=75, mayy=70, yin=80, vinay=69)
print(a)

# from a list of tuples
a = dict([('ilia', 54), ('hao', 64), ('arben', 75), ('mayy', 70),
('yin', 80), ('vinay', 69)])
print(a)

# using dictionary comprehension
a = {name: score for name, score in [('ilia', 54), ('hao', 64),
('arben', 75), ('mayy', 70), ('yin', 80), ('vinay', 69)]}
print(a)

# dict comprehension
import random
names = 'hao ilia mayy arben yin narendra vinay'.split()

b = { name : [random.randint(45,100) for _ in range(5)] for name in
names}
for (name, scores) in b.items():
    print(f'{name:>8}: {scores}')

# from zip()
a = dict(zip(['ilia', 'hao', 'arben', 'mayy', 'yin', 'vinay'], [54, 64,
75, 70, 80, 69]))
print(a)

# using the fromkeys() method
default_value = 50
a = dict.fromkeys(['ilia', 'hao', 'arben', 'mayy', 'yin', 'vinay'],
default_value)
print(a)

# key can be any immutable type: int, string, tuple
# of course the value can be anything!
c = {1: 'one', 'two' : 2, (3,) : 'three', 4 : [0, 1, 2, 3]}
print(c)
```

Lecture 07

```
for (key, val) in c.items():
    print(f'{key}: {val}')
```

Some common operation

- using the len() function to obtain the number of pairs
- using a key to read a value
- using a key to set/update a value
- using a key to delete a pair
- Adding a new key-value pair
- Adding multiple key-value pairs

Using the len() function to obtain the size of a dictionary

```
a = {'ilia': 54, 'hao': 64, 'arben' : 75, 'mayy' : 70, 'yin' : 80,
     'vinay' : 69}
```

```
#accessing a value using a key
print(f'There are {len(a)} pairs in the dictionary')
```

Using a key to read or access a value

```
a = {'ilia': 54, 'hao': 64, 'arben' : 75, 'mayy' : 70, 'yin' : 80,
     'vinay' : 69}
```

```
#accessing a value using a key
key = 'arben'
print(f'the key "{key}" has the value {a[key]}')
```

Check if a key exist

```
a = {'ilia': 54, 'hao': 64, 'arben' : 75, 'mayy' : 70, 'yin' : 80,
     'vinay' : 69}
```

```
key = 'arben'
print(f'the key "{key}" exists -> {key in a}')
```

```
key = 'narendra'
print(f'the key "{key}" exists -> {key in a}')
```

Using a key to set/update a value

```
#setting a value using a key
key = 'arben'
print(f'before: \'{key}\'' -> {a[key]}')
```

```
value = 100
a[key] = value                                # if the key does not exist,
                                              # a new key-value pair will be added to
the dict
print(f' after: \'{key}\'' -> {a[key]}')
```

Using a key to delete a pair

Lecture 07

```
#deleting a key-value pair using a key
# executing this cell will raise an error, because in the second run,
the key 'ilia' has been deleted
# and the dictionary no longer contains it
key = 'ilia'
del a[key]
print(a)
# adding a new key-value pair
key = 'narendra'
a[key] = 55
print(a)
```

Adding multiple key-value pairs

```
a.update({'mehrdad' : 90, 'justin' : 45})
print(a)
```

Accessing a value

- You may use the key to access a particular value
- A better way to access the value associated with a key is via the `get()` method

```
#using the key to access a value
key = 'justin'
print(f'{key} score is {a[key]}')

# if the key does not exist, it will raise a KeyError
# so it is better to use the get() method
key = 'justine'
print(f'{key} score is {a.get(key, 0)}')    #if the key does not exist,
then the argument is returned
```

Printing all the keys

```
for k in a.keys():
    print(k, end=' ')
```

Printing all the values

```
for val in a.values():
    print(val, end=' ')
```

Iteration the dict

Printing both the keys and the values

```
print('method 1')
for (name, score) in a.items():
    print(f'{name:>8}: {score}')
```

```
#method 2
print('\nmethod 2')
for key in a:
    print(f'{key:>8}: {a[key]}')
```


Nested dictionary

You can store dictionaries inside dictionaries

```
students = {  
    'hao': {  
        'scores': [85, 92, 78],  
        'grade': 'A',  
        'attendance': 95  
    },  
    'ilia': {  
        'scores': [76, 88, 92],  
        'grade': 'B+',  
        'attendance': 88  
    },  
    'mayy': {  
        'scores': [92, 95, 98],  
        'grade': 'A+',  
        'attendance': 100  
    }  
}  
  
print(students['hao'])          # Output: {'scores': [85, 92, 78],  
                                'grade': 'A', 'attendance': 95}  
print(students['hao']['scores']) # Output: [85, 92, 78]  
print(students['hao']['scores'][0]) # Output: 85
```

Summary

- A dictionary is a flexible, efficient way to store and access key-value pairs.
- A dictionary maps unique keys to values.
- Keys must be hashable; values can be any type.
- Common operations: access with key, get(), update(), pop(), del.
- Great for fast lookups, structured data, and mapping relationships.

All the collection type

It is possible to convert from one collection to another

Collection

├ Set-like (unordered, unique elements)

| ├ set, frozenset

|

├ Sequence (ordered)

| ├ list, tuple, str, range

|

└ Mapping (key-value)

 └ dict

Contents:

1. [Set](#) to other collections
2. [String](#) to other collections
3. [Tuples](#) to other collections
4. [List](#) to other collections
5. [Dictionary](#) to other collections
6. [Functions](#) associated with collections
 - str.join()
 - dict.fromkeys()
 - eval()
 - join()
 - zip()
7. [Summary](#)

Sets to other types

Lecture 07

Since the set collection maintains uniqueness, duplicates are silently removed when converting from other collections to set.

```
s = {'one', 'two', 'three', 'one', 'four' }
print(f'set (ORIG): {s}')          # notice the order changes and the
duplicate 'one' is removed

# to string
a = str(s)
print(f'      str: {a}')

# to tuple
a = tuple(s)
print(f'      tuple: {a}')

# to list
a = list(s)
print(f'      list: {a}')

# to dictionary
# direct conversion from set to dictionary is not possible
# see the fromkey() method of the dict class
print(f'      dict: direct conversion from set is not possible')
```

String to other types

```
s = 'one two three one four'
print(f'      (ORIG)\n      string: {s}')

# to set
b = set(s)
print(f'      set: {b}')

# to tuple
b = tuple(s)
print(f'      tuple: {b}')

# to list
a = list(s)
print(f'      list: {a}')

# to dict
# see the fromkey() method of the dict class
print(f'      dict: direct conversion from string is not possible')
```

Tuples to other types

```
tup = ('one', 'two', 'three', 'one', 'four' )
print(f'tuple (ORIG): {tup}')

# to set
a = set(tup)
print(f'      set: {a}')
```

Lecture 07

```
# to list
a = list(tup)
print(f'          list: {a}')
```



```
# to string
a = str(tup)
print(f'          str: {a}')
```



```
# to dictionary
# direct conversion from tup to dictionary is not possible
# see the fromkey() method of the dict class
print('          dict: direct conversion from tuple is not possible')
```

List to other types

```
lst = ['one', 'two', 'three', 'one', 'four' ]
print(f'          List: {lst}')
```



```
# to set
a = set(lst)
print(f'          Set: {a}')
```



```
# to tuple
b = tuple(lst)
print(f'          Tuple: {b}')
```



```
# to string
c = ' '.join(lst)
print(f'          String: {c}')
```



```
# to dictionary
# direct conversion from list to dictionary is not possible
# see the fromkey() method of the dict class
print('          dict: direct conversion from list is not possible')
```

Dictionary to other types

Because a dict is a collection of key-value pairs, most conversions will only transfer the keys!

```
dic = {1: 'one', 2: 'two', 3: 'three', 4: 'four' }
print(f'(ORIG)\nDictionary: {dic}')
```



```
# to set
# returns the keys of the dictionary as a set
a = set(dic)
print(f'          Set: {a}')
```



```
# to string
# returns the entire dictionary as a string
```

Lecture 07

```
c = str(dic)
print(f'    String: {c}')

# to tuple
# returns the keys of the dictionary as a tuple
b = tuple(dic)
print(f'    Tuple: {b}')

# to list
# returns the keys of the dictionary as a list
d = list(dic) # using set elements as keys with a default value
print(f'    List: {d}')
```

Functions associated with collections

```
# collections definition for this section
#set
a = {'one', 'two', 'three', 'one', 'four' }

# string
b = 'one two three one four'

# tuple
c = ('one', 'two', 'three', 'one', 'four' )

# list
d = ['one', 'two', 'three', 'one', 'four' ]

# dictionary
e = {1: 'one', 2: 'two', 3: 'three', 4: 'four' }
```

join() method from the string class

```
# the str.join() function from the string class
# joins elements of a collection into a string with a specified separator
# the separator is a comma and a space
# works with set, tuple, and list but not with dictionary directly
print(f'    set: {"", ".join(a)}')
print(f'    str: {"", ".join(b)}')
print(f'    tuple: {"", ".join(c)}')
print(f'    list: {"", ".join(d)}')

# this join() function does not work with dictionaries directly
# but we can join the keys or values separately if we choose
print('    dict: join() does not work')
```

fromkeys() method from the dict class

```
# the dict.fromkeys() method from the dict class
# creates a new dictionary with keys from the given iterable and a default value
# the iterable can be a set, string, tuple, or list but not a dictionary
# the default value is specified as the second argument, if not provided,
# the default value is None
```

Lecture 07

```
#
print(f'          set: {dict.fromkeys(a, 0)}')
print(f'          str: {dict.fromkeys(b, 1)}')
print(f'        tuple: {dict.fromkeys(c, 2)}')
print(f'          list: {dict.fromkeys(d, 3)}')
print(f'          dict: {dict.fromkeys(e, 4)}')
```

eval() function

This is a strange function, it takes a string and evaluates it like it would at the repl prompt

```
# set
a = "{ 'one', 'two', 'three', 'one', 'four' }"

# string
b = "'one two three one four'"

# tuple
c = "('one', 'two', 'three', 'one', 'four' )"

# list
d = "['one', 'two', 'three', 'one', 'four' ]"

# dictionary
e = "{1: 'one', 2: 'two', 3: 'three', 4: 'four' }"

print(f'          set: {eval(a)}')
print(f'          str: {eval(b)}')
print(f'        tuple: {eval(c)}')
print(f'          list: {eval(d)}')
print(f'          dict: {eval(e)}')
```

The split() method of the str class

```
# the split() method from the string class
# splits a string into a list of substrings based on a specified separator
# the default separator is any whitespace
# works only with strings

s = 'one two three one four'
print(f'(ORIG) string: {s}')

print(f'          list: {s.split()}')
```

The zip() function()

zip() pairs up elements from multiple iterables into tuples, stopping at the shortest and returns an iterator of the tuples

```
a = [0, 1, 2]
b = {'0', '1', '2', '3'}
c = ('zero', 'one', 'two')
```

Lecture 07

```
# the zip() function pairs up elements from multiple iterables into tuples,
stopping at the shortest and returns an iterator of the tuples
zipped = zip(a, b, c)
print(f'        list: {list(zipped)}') # convert the iterator to a list to see
the result

zipped = zip(a, b, c)
print(f'        tuple: {tuple(zipped)}') # convert the iterator to a list to
see the result

zipped = zip(a, b, c)
print(f'        set: {set(zipped)}') # convert the iterator to a list to see
the result

zipped = zip(b, c)
print(f'        dict: {dict(zipped)}') # convert the iterator to a list to see
the result
```

You can unzip with the *operator

```
zipped = zip(a, b, c)
a, b, c = zip(*zipped) # unzip the zipped object back into individual
iterables
print(f'        a: {a}')
print(f'        b: {b}')
print(f'        c: {c}')
```

Comparison of the collections

Sequence Methods directly associated with the class

set	add, clear, copy, difference, difference_update, discard, intersection, intersection_update, isdisjoint, issubset, issuperset, pop, remove, symmetric_difference, symmetric_difference_update, union, update
dict	clear, copy, fromkeys, get, items, keys, pop, popitem, setdefault, update, values

Comparison of the common non-primitive built-in types

	Iterable	Indexable	Collection	Sequence
list	yes	yes	yes	yes
tuple	yes	yes	yes	yes
str	yes	yes	yes	yes
range	yes	yes	yes	yes

Lecture 07

	Iterable	Indexable	Collection	Sequence
dict	yes	yes	yes	no
set	yes	no	no	no
file_obj	yes	no	no	no

Summary

- Iterable → can be looped over (for x in ...).
- Sequence → an ordered iterable with indexing/slicing (list, tuple, str, range).
- Collection → general container data structures (including sequences, sets, dicts).

```
dat = {0:0, 1:'one', 2:[2, 3]}  
print(dat.keys())
```

```
print(type(dat.keys()))
```

```
for k in dat.keys():  
    print(k, end=' ')  
print()
```

```
print()  
for k in dat.values():  
    print(k, end=' ')
```


Comprehension vs. Generators in Python

In Python, comprehensions and generators are powerful tools for creating and managing sequences of data efficiently and concisely.

Contents:

1. [Comprehension](#)
 - Set Comprehension
 - List Comprehension
 - Dictionary Comprehension
2. [Generator](#)
 - Generator Functions
 - Generator Expressions
3. [Summary](#)

Comprehension

Comprehensions provide a concise, readable way to create new collections like lists, sets, or dictionaries.

Benefits:

- Compact and expressive syntax
- Often faster than equivalent loops
- Supports conditional filtering

Set comprehension

Creates a new unordered collection of unique items.

Syntax: {expression for item in iterable if condition}

Set of word length

```
# example of a set comprehension
unique_lengths = {len(word) for word in ['apple', 'banana', 'cherry']}
print(unique_lengths)
```

First letter of names

```
# another example
names = ['Alice', 'Bob', 'Charlie', 'David']
first_letters = {name[0] for name in names}
print(first_letters)
```

With condition

Lecture 07

```
# example with a condition
even_numbers = {num for num in range(10) if num % 2 == 0}
```

Using a function

```
# example of a set comprehension with a function
def square(x):
    return x * x
```

```
squared_set = {square(x) for x in range(5)}
print(squared_set)
```

With multiple iterables

```
# a more complex example
# using multiple iterables

complex_set = {f'{x * y}' for x in range(1, 4) for y in range(1, 4)}
print(complex_set)
```

List comprehension

List comprehension create new lists by applying an expression to each item in an iterable, optionally filtering items.

The Basic Form

```
[expression for item in iterable if condition]
```

where

- expression: what you want in the list
- item: each element from the iterable
- condition: optional filter

```
# example of a list comprehension
evens = [x for x in range(21) if x % 2 == 0]
print(evens)
```

Using more than one iterable

```
# another example
colors = ['red', 'green', 'blue']
vehicles = ['car', 'bus']
combinations = [f'{color} {obj}' for color in colors for obj in vehicles]
print(combinations)
print([f'{a}{b}{c}' for a in 'ab' for b in '12' for c in 'XYZ'])
```

Using a function

```
# Apply function to elements
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
```

Lecture 07

```
prime_squares = [x**2 for x in range(20) if is_prime(x)]
print(prime_squares)
```

Flattening a 2-D list

```
# flattening a 2-D list (a list of lists)
nested_list = [[1, 2, 3], [4, 5, 6, 7], [8, 9]]
flattened_list = [item for sublist in nested_list for item in sublist]
print(flattened_list)
```

Dictionary comprehension

The Basic Form

```
{key_expr: value_expr for item in iterable if condition}
where
```

- key_expr: Expression for the key
- value_expr: Expression for the value
- iterable: Any iterable (like a list, tuple, or range)
- condition: Optional filter

```
# example of a set comprehension
words = ['apple', 'banana', 'cherry']
word_lengths = {word: len(word) for word in words}
print(word_lengths)
```

```
# another example
num_to_str = {x: f'Number {x}' for x in range(5)}
print(num_to_str)
```

```
# another example
original = {'a': 1, 'b': 2, 'c': 3}
inverted = {value: key for key, value in original.items()}
print(inverted)
```

Generators

A generator is a special type of iterator that yields items one at a time using the yield keyword. It doesn't store the entire sequence in memory, making it ideal for large datasets or infinite sequences.

Benefits:

- Memory-efficient: Generates items on-the-fly.
- Lazy evaluation: Only computes values when needed.
- Useful for pipelines: Can be chained together.

Cons:

- Can't index or re-iterate without re-creating it.
- Once exhausted, it's gone.

Lecture 07

- Can only be used once

Generator Function

A generator in Python is a special kind of function that returns an iterator, allowing you to generate a sequence of values one at a time, on demand, without storing them all in memory.

Generator functions are defined like regular functions but use the `yield` statement instead of `return`. Every time the generator's next value is requested (using `next()` or a `for` loop), execution resumes right where it left off after the last `yield`, maintaining the function's local state.

```
def count_up_to(n):  
    count = 1  
    while count <= n:  
        yield count  
        count += 1  
  
for number in count_up_to(5):  
    print(number, end=' ')
```

Generator Expression

Generator expressions, are similar to list comprehensions, but use parentheses `()` instead of brackets `[]` and create generator objects rather than lists.

```
# defining a generator expression  
squares = (x*x for x in range(10))  
# consuming the generator  
for square in squares:  
    print(square, end=' ')  
  
# running the generator expression again will not yield any results  
for square in squares:  
    print(square, end=' ')
```

Summary:

Feature	Comprehensions	Generators
Output Type	List / Set / Dict	Iterator
Memory Usage	Stores all elements	Generates on demand
Reusability	Can be reused	One-time use

Lecture 07

Feature	Comprehensions	Generators
Speed	Faster for small data	Better for large/infinite data
Syntax Example	<code>[x**2 for x in range(5)]</code>	<code>(x**2 for x in range(5))</code>

1. Use comprehensions when:

- You need to access the data multiple times
- You need list/dict/set methods
- The dataset is small

2. Use generators when:

- Working with large datasets
- You only need to iterate once
- Memory efficiency is important
- Processing data in pipelines